



Universidade do Minho
Escola de Engenharia

Licenciatura em Engenharia Informática

Processamento de Linguagens

Grupo 14

Relatório Final Trabalho Prático
Compilador de Fortran 77 para EWVM

Gonçalo Barbosa A104439
Gonçalo Carmo A104619
Simão Pereira A104535

17 de Maio 2026

1. Introdução

O presente relatório documenta a arquitetura e desenvolvimento de um compilador capaz de traduzir código da linguagem Fortran 77 para código *Assembly* destinado a execução numa Máquina Virtual baseada em pilha que foi disponibilizada pela equipa docente (EWVM). O projeto foi inteiramente desenvolvido em Python, recorrendo à biblioteca PLY para a construção do analisador léxico (*ply.lex*) e sintático (*ply.yacc*). A arquitetura do compilador segue o seguinte modelo:

1. **Análise Léxica (*Lexer*):** Tokenização do código fonte.
2. **Análise Sintática (*Parser*):** Geração da Árvore Sintática Abstrata (AST).
3. **Análise Semântica (*Semantic*):** Validação rigorosa de tipos, variáveis e escopos.
4. **Otimização de AST (*Optimizer*):** Passagem intermédia para maximização de eficiência.
5. **Geração de Código (*Codegen*):** Emissão das instruções EWVM.

2. Instruções de Execução do Compilador

Para facilitar o uso e garantir uma integração simples na linha de comandos, o compilador foi encapsulado num ficheiro orquestrador principal designado `main.py`.

Pré-requisitos:

- Python 3 instalado no sistema operativo.
- Biblioteca PLY instalada.

Sintaxe Base de Utilização:

```
python main.py <ficheiro_fonte.f> [-o <ficheiro_saida.vm>] [-O]
```

Parâmetros e Flags Disponíveis:

- `<ficheiro_fonte.f>`: Caminho para o ficheiro contendo o código em Fortran 77. Este é o único argumento obrigatório.
- `-o` ou `--output`: Flag opcional. Especifica o nome e o caminho do ficheiro a ser gerado com as instruções EWVM. Caso seja omitido, o compilador gera automaticamente um ficheiro com o mesmo nome do ficheiro fonte, substituindo a extensão original para `.vm`.
- `-O` ou `--optimize`: Flag opcional. Ativa o módulo de otimização de código, efetuando *Constant Folding* e *Dead Variable Elimination* na Árvore Sintática antes da geração de código.

Ferramenta de testes (`test.py`):

Para além do orquestrador principal, o projeto inclui um *script* auxiliar desenhado especificamente para o desenvolvimento iterativo, permitindo a inspeção rigorosa de cada fase independente da *pipeline* de compilação. Através da sintaxe `python test.py <ficheiro_fonte.f> [modo]`, é possível isolar componentes utilizando um conjunto de *flags* mutuamente exclusivas:

- `-lexer`: Processa o código e imprime o fluxo sequencial de *tokens* detetados, permitindo validar o reconhecimento de identificadores e a correta aplicação de expressões regulares.
- `-parser`: Limita a execução à validação gramatical LALR(1). Se não houver conflitos, imprime a confirmação “Syntax OK”.
- `-ast`: Imprime uma representação visual hierárquica, estruturada e indentada da Árvore Sintática Abstrata. Foi uma ferramenta indispensável para validar a precedência de operadores e o achatamento estrutural do ciclo DO.
- `-semantic`: Executa a leitura da AST e a consequente validação rigorosa de tipos e escopos, assegurando o preenchimento correto da Tabela de Símbolos.
- `-codegen`: Processa a *pipeline* na íntegra e imprime as instruções *Assembly* da EWVM diretamente no terminal (*stdout*). Esta abordagem omite a gravação em disco, permitindo aos programadores uma inspeção visual imediata do código gerado durante a validação de novos testes.

3. Análise Léxica (Lexer)

A primeira etapa do compilador é responsável por ler o código-fonte em texto simples e transformá-lo numa sequência de *tokens* válidos para o Parser, utilizando o módulo `ply.lex`. A construção do Lexer exigiu várias opções de *design* para adaptar a linguagem histórica aos paradigmas modernos de compilação.

3.1. Adoção de Formato Livre (*Free-Form*)

Tradicionalmente, o Fortran 77 exige um formato fixo e posicional (colunas 1-5 reservadas para *labels*, coluna 6 para marcação de continuação de linha, e colunas 7-72 para instruções). No enunciado deste projeto foi-nos dada a possibilidade de optar pelo formato tradicional descrito ou pelo formato mais moderno, conhecido por formato livre (*Free-Form*). Para este projeto, tomou-se a decisão arquitetural de implementar o formato livre (*Free-Form*). O Lexer foi instruído a ignorar espaços e tabulações (`t_ignore = " \t"`), processando o código como um fluxo contínuo de caracteres independentemente da coluna onde se encontram. Esta abordagem simplifica drasticamente a construção da gramática LALR no PLY e moderniza a experiência de escrita do utilizador, abdicando da rigidez histórica herdada dos cartões perfurados.

3.2. Estrutura de Tokens e Expressões Regulares

O Lexer define um conjunto rigoroso de regras léxicas implementadas através de expressões regulares (Regex), garantindo que apenas construções válidas prosseguem para a fase sintática:

- **Identificadores e *Case-Insensitivity*:** As expressões regulares foram configuradas para aceitar apenas identificadores alfanuméricos começados por letras (`[A-Za-z][A-Za-z0-9]*`). O Lexer rejeita ativamente caracteres como o *underscore* (`_`), garantindo precisão histórica. Para respeitar o *standard* do Fortran que dita que a linguagem é *case-insensitive*, a função `t_ID` efetua uma normalização automática, convertendo todos os identificadores capturados para maiúsculas (ex., `Soma` é lida e registada na Tabela de Símbolos como `SOMA`). É nesta conversão que o Lexer verifica se o identificador pertence ao dicionário de palavras reservadas (como `PROGRAM`, `INTEGER`, `DO`, etc.).
- **Operadores Nativos Fortran:** Os operadores relacionais e lógicos, que em Fortran possuem uma sintaxe baseada em pontos (ex., `.EQ.`, `.AND.`, `.NOT.`), foram capturados por expressões regulares exatas e convertidos em *tokens* unívocos para evitar ambiguidades com chamadas de funções matemáticas.
- **Literais Numéricos:** Foram criadas funções de captura distintas para Inteiros e Reais. A expressão `r"(\d+\.\d*|\.\d+)"` garante a captura correta de números de vírgula flutuante (mesmo com omissão do zero inicial, como `.5`).
- **Manipulação de Strings:** O suporte para *strings* engloba o tratamento de plicas simples (`'`). A expressão `r"'([\^'\n]|'')*'"` não só captura a *string*, como lida inteligentemente com a sintaxe de escape interna do Fortran (onde duas plicas seguidas representam uma plica literal no interior da *string*), utilizando o método `replace` do Python durante a geração do *token*.

3.3. Controlo de Erros e Linhas

A contagem de quebras de linha (`\n`) foi rigorosamente mantida através da função `t_newline`. Isto permite que, caso ocorra uma falha léxica ou sintática, o compilador reporte não só o erro, mas a linha exata onde ocorreu, facilitando a depuração por parte do utilizador.

4. Análise Sintática e a Gramática Utilizada

A Análise Sintática tira partido do analisador LALR(1) gerado pelo `ply.yacc`. A gramática livre de contexto desenhada para este projeto foi estruturada para garantir a ausência total de conflitos *Shift/Reduce*. Isto foi alcançado através da utilização rigorosa de listas recursivas à esquerda e da definição explícita de uma tabela de precedências para os operadores.

4.1. Precedência e Associatividade de Operadores

Para evitar ambiguidades na resolução de expressões matemáticas e lógicas, definiu-se a tupla **precedence** nativa do `PLY`. A ordem foi estabelecida da menor para a maior precedência, ditando rigorosamente como o *parser* agrupa os *tokens*:

1. **Lógicos (Menor precedência):** OR e AND (associativos à esquerda).
2. **Negação Lógica:** NOT (associativo à direita).
3. **Relacionais:** .EQ., .NE., .LT., .LE., .GT., .GE. foram marcados como **não-associativos** (nonassoc). Esta foi uma decisão de segurança crucial: impede construções sintaticamente inválidas no Fortran como `A .EQ. B .EQ. C`, forçando o *parser* a atirar um erro de sintaxe a menos que o utilizador isole as operações com parênteses ou operadores lógicos.
4. **Aritmética Base:** Soma (PLUS) e Subtração (MINUS) (associativos à esquerda).
5. **Aritmética de Fatores:** Multiplicação (TIMES) e Divisão (DIVIDE) (associativos à esquerda).
6. **Unário (Maior precedência):** O sinal de negativo unário foi definido como associativo à direita com a etiqueta `UMINUS`. Para que o *parser* saiba distinguir o “menos unário” do “menos de subtração”, aplicou-se a diretiva `%prec UMINUS` diretamente na regra gramatical, forçando a sua precedência máxima.

4.2. Especificação da Gramática

A gramática adotada flui de forma hierárquica e contínua desde o nó raiz (`program`), passando pelas definições de variáveis (`declarations`), até atingir as instruções executáveis (`statement_list`) e as suas respetivas expressões lógicas e aritméticas.

```
program : PROGRAM ID declarations statement_list END
```

```
declarations : declarations declaration  
             | empty
```

```
declaration : type_spec declarator_list
```

```
type_spec : INTEGER  
          | REAL  
          | LOGICAL
```

```
declarator_list : declarator_list COMMA declarator  
               | declarator
```

```
declarator : ID  
           | ID LPAREN expression RPAREN
```

```
statement_list : statement_list statement  
               | empty
```

```
statement : optional_label statement_body
```

```
optional_label : INTEGER_LITERAL  
              | empty
```

```
statement_body : assign_statement  
               | print_statement  
               | read_statement
```

```

        | if_statement
        | do_header_statement
        | goto_statement
        | continue_statement

assign_statement : variable_ref ASSIGN expression

if_statement : IF LPAREN expression RPAREN THEN statement_list else_part ENDIF

do_header_statement : DO INTEGER_LITERAL ID ASSIGN expression COMMA expression

expression : expression PLUS expression
            | expression TIMES expression
            | MINUS expression %prec UMINUS
            | NOT expression
            | LPAREN expression RPAREN
            | INTEGER_LITERAL
            | REAL_LITERAL
            | variable_ref
            | REAL LPAREN expression_list RPAREN

variable_ref : ID
             | ID LPAREN expression_list RPAREN

```

4.3. O Ciclo DO

A principal dificuldade arquitetural no *Parser* surgiu como resposta à implementação do ciclo DO no Fortran. Na maioria das linguagens, um ciclo contém um bloco aninhado (ex., { ... }). Em Fortran, o ciclo DO e a sua instrução de fecho CONTINUE são comandos sintaticamente separados, operando quase como um *goto* condicional. A tentativa de forçar uma árvore hierárquica (tentar colocar as instruções fisicamente dentro de um nó DoLoop no *Parser*) revelou-se impeditiva. Assim, o *Parser* foi concebido para ler o cabeçalho (DoHeader) e o fecho (Continue) como instruções independentes. A responsabilidade de garantir que estas peças encaixam foi transferida para a análise semântica.

4.4. Resolução de Ambiguidade: Arrays vs Funções Intrínsecas

Em Fortran, a sintaxe de acesso a um índice de um array (ex., NUMS(I)) e a chamada de uma função (ex., MOD(A, B)) é idêntica. O *Parser* foi instruído a não tentar desambiguar esta situação de imediato, gerando sempre um nó genérico `ArrayRef` na Árvore Sintática. A validação de que se trata de uma chamada válida à biblioteca standard é posteriormente resolvida pela análise semântica.

A única adaptação necessária foi para a função REAL(X) que converte números inteiros em números reais (*float*). Uma vez que a palavra REAL é um *token* reservado para a declaração de tipos, o *Parser* rejeitaria o código. Adicionou-se uma regra excecional para permitir que o *token* reservado REAL seja processado como uma expressão funcional quando seguido de parênteses (*expression* : REAL LPAREN *expression_list* RPAREN).

4.5. Representação Intermédia: Árvore Sintática Abstrata (ast_nodes.py)

Em vez de proceder à tradução imediata do código-fonte para linguagem máquina, a conclusão da fase de análise sintática resulta na geração de uma Árvore Sintática Abstrata (AST). O módulo `ast_nodes.py` estabelece o modelo de dados rigoroso desta representação intermédia, mapeando cada construção gramatical do subconjunto Fortran 77 para uma classe dedicada em Python (e.g., `Program`, `IfThenElse`, `BinOp`, `DoHeader`).

A centralização da AST neste ficheiro assenta em três pilares arquiteturais que garantem a escalabilidade do compilador:

1. **Desacoplamento Arquitetural:** O uso de classes independentes para cada nó garante uma separação estrita de responsabilidades. O *Parser* foca-se exclusivamente no reconhecimento sintático

e na instanciação destes objetos, isolando-se por completo da complexidade da validação semântica e da futura emissão de *Assembly*.

2. **Viabilização do Padrão *Visitor*:** Ao estabelecer uma hierarquia de classes fortemente tipada, o modelo de dados permite que as fases subsequentes da *pipeline* (Análise Semântica, Otimizador e Gerador de Código) apliquem o padrão de desenho *Visitor*. Isto possibilita a travessia recursiva da árvore.
3. **Encapsulamento de Estado e Contexto:** As instâncias destas classes atuam como contentores seguros de metadados. Informações críticas — como os identificadores numéricos de *labels* num ciclo D0, as referências de uma variável, ou a hierarquia esquerda-direita de uma expressão matemática — são encapsuladas e transportadas de forma imutável através de todas as fases de compilação até ao mapeamento final para a EWVM.

5. Análise Semântica

A etapa semântica varre a AST gerada para garantir a correção lógica e tipológica do programa, atuando como o principal mecanismo de defesa da Máquina Virtual. Se uma anomalia for detetada, é lançada a exceção `SemanticError`.

Validações Implementadas:

- **Tabela de Símbolos (*Symbol Table*):** É exigida a declaração prévia de todas as variáveis. O analisador constrói um registo rigoroso que mapeia cada identificador ao seu tipo de dados original (`INTEGER`, `REAL`, `LOGICAL` ou *Array*). Esta “tabela de tipos” é crucial não só para a validação estrita das expressões, mas também para ser consultada mais tarde pelo gerador de código. É também proibido utilizar *arrays* globais sem a correspondente indexação explícita.
- **Pré-Validação de *Labels*:** A árvore é varrida integralmente para recolher todos os *labels* declarados. Isto impede que a instrução `GOTO` tente saltar para zonas inexistentes.
- **Isolamento de Escopos:** Para validar o ciclo D0 é analisada apenas a lista de instruções (*statement_list*) onde este se encontra. Este *design* garante que o código não cruze blocos de execução (por exemplo, iniciar um D0 no bloco principal e terminá-lo ilegalmente no interior de um bloco `IF`).
- **Segurança de Tipos (*Type Safety*):** A EWVM não realiza promoção automática de tipos. A injeção de um valor inteiro numa operação `fadd` (Float Add) resultaria numa falha. O analisador semântico blinda o sistema exigindo coerência total entre operandos e, especificamente para funções como `INT()`, exige estritamente argumentos `REAL` (e vice-versa).

6. Otimização Intermédia de Código (AST)

Como mecanismo de valorização e eficiência, foi desenvolvido um módulo otimizador (`optimizer.py`), ativado pela flag `-O`. Este atua modificando a AST *in-place* antes da geração de código.

Passagem 1: Dobragem de Constantes (*Constant Folding*) Utilizando um algoritmo *bottom-up*, expressões determinísticas são pré-calculadas. Por exemplo, a representação em árvore de $10 * 5 + 2$ é matematicamente colapsada num único nó de valor 52. Para evitar falhas internas no próprio compilador, o otimizador possui proteções contra divisões por zero, delegando essa responsabilidade para a execução final (*runtime*).

Passagem 2: Eliminação de Variáveis Mortas (*Dead Code Elimination*) O algoritmo efetua duas leituras: primeiro varre todas as variáveis efetivamente utilizadas nas expressões; de seguida, varre as declarações iniciais do programa. Qualquer variável declarada que não figure na lista de utilização é suprimida da AST, poupando assim instruções de alocação de memória na máquina virtual.

7. Geração de Código para EWVM (*Backend*)

A fase final e mais complexa do projeto consiste na tradução da Árvore Sintática Abstrata (AST) otimizada num conjunto linear de instruções compatíveis com a arquitetura *Stack-Based* da Máquina Virtual disponibilizada (EWVM). O módulo `codegen.py` implementa um padrão *Visitor* que percorre a árvore recursivamente e emite os respetivos *opcodes*.

7.1. Gestão de Memória e Tabela de Tipos

Uma vez que não foram implementadas sub-rotinas dinâmicas com escopos locais, o modelo de memória assenta num ambiente estruturado globalmente. O gerador gere internamente um contador de endereços (`next_address`) e dicionários de associação cruciais: o `address_map` (que liga o nome da variável à sua posição física) e a **Tabela de Tipos** (`type_map`), que herda o conhecimento registado na fase Semântica.

- **Variáveis Escalares:** São alocadas sequencialmente no *Global Pointer* (`gp`). Na declaração, o compilador regista o seu tipo na Tabela de Tipos, e emite um `pushi 0` seguido de `storeg N`, reservando o espaço.
- **Arrays (Vetores):** Tiram partido do *Heap* da EWVM. O gerador avalia a expressão de tamanho do *array*, emite a instrução de alocação dinâmica `allocn`, guarda o endereço de base resultante no *Global Pointer* (`storeg N`) e regista a variável como um *array* na Tabela de Tipos.

7.2. O Paradigma de Indexação do Fortran

Uma das particularidades históricas da linguagem Fortran é o facto de a indexação nativa de *arrays* começar no índice 1, enquanto a vasta maioria das arquiteturas físicas e virtuais modernas (incluindo a EWVM) utiliza indexação começada em 0. Para resolver este problema, é efetuada uma tradução em tempo real. Sempre que o nó `ArrayRef` é visitado (para leitura via `loadn` ou escrita via `storen`), o compilador acede ao endereço base no `gp`, avalia a expressão do índice na pilha e injeta automaticamente as instruções `pushi 1` e `sub`. Este desvio (*offset*) garante o mapeamento perfeito entre a sintaxe do FORTRAN e a alocação da EWVM.

7.3. Inferência de Tipos Dinâmica e Casting Implícito

A EWVM possui instruções matemáticas e de I/O estritamente tipadas. O gerador de código implementa inferência de tipos em tempo real (`expr_type`) para decidir qual a via de execução correta:

- **Operações Aritméticas e Unárias:** Para a mesma operação (e.g., adição), o compilador avalia os nós filhos e decide se deve emitir `add` ou `fadd`, `mul` ou `fmul`. O operador unário negativo (`-X`) foi resolvido emitindo uma multiplicação por `-1` (para inteiros) ou `-1.0` (para reais).
- **Casting Implícito na Pilha:** O Fortran suporta matemática mista. Ao percorrer expressões como `FLOAT_VAR = INT_VAR * 2.5`, o compilador deteta a disrupção de tipos e injeta estrategicamente a instrução `itof` (*Integer to Float*) na pilha, antes da execução do operador genérico de reais.
- **Input / Output (I/O):** As instruções de `PRINT` e `READ` interrogam a tabela de tipos e emitem os *opcodes* adequados consoante o destino (`writes`, `writai`, `writef` para saída, e conversores `atoi`, `atof` pós-leitura de consola). Adicionalmente, as *Strings* sofrem um processamento de *escape* dinâmico antes da emissão para garantir a estabilidade da máquina virtual. A título de exemplo, se o código FORTRAN contiver uma instrução com aspas embutidas, como `PRINT *, 'Ele disse "Ola"'`, a geração de *Assembly* resultaria em `pushs "Ele disse "Ola"`. Isto faria com que a EWVM interpretasse prematuramente o fecho da *String* quando encontrasse a primeira aspa em `"Ola"`, colapsando com um erro de sintaxe. Para prevenir este cenário, o compilador insere ativamente caracteres de escape no interior da *String*, gerando a instrução correta `pushs "Ele disse \"Ola\""`.

7.4. Estruturas de Controlo e o Motor do Ciclo DO

A tradução de estruturas de controlo exige uma gestão cuidadosa de saltos na Máquina Virtual. A ramificação condicional simples (`IfThenElse`) foi implementada através da emissão dinâmica de *labels* únicos e sequenciais (e.g., `ELSE1`, `ENDIF1`). O compilador avalia a condição (que deixa 0 ou 1 no topo da pilha) e usa a instrução `jz` (*Jump if Zero*) para saltar para o bloco `ELSE` caso a condição seja falsa. No final do bloco `THEN`, é emitido um salto incondicional (`jump`) para o `ENDIF`, impedindo a execução do código alternativo.

No entanto, o maior desafio de engenharia na geração de código consistiu em resolver a compilação do ciclo `DO`.

O Paradigma do Achatamento da AST Em linguagens modernas, um ciclo é um bloco aninhado. Em Fortran 77, um ciclo é definido por duas instruções fisicamente separadas que partilham um identificador numérico (o *label*). Exemplo:

```
DO 10 I = 1, 5
PRINT *, I
10 CONTINUE
```

Como explicado na Análise Sintática, a nossa AST não agrupa o `PRINT` dentro do `DO`. Em vez disso, a AST contém uma lista plana de três nós independentes: `[DoHeader, Print, Continue]`.

O problema que se colocava na Geração de Código era: Como é que o nó `Continue`, ao ser visitado depois, sabe qual é a variável que tem de incrementar e para onde tem de saltar de volta?

A Solução: O Dicionário de Contexto (`loop_contexts`) Para resolver este problema, o *codegen* atua como uma máquina de estados, utilizando um dicionário interno chamado `loop_contexts`. A tradução de um ciclo decorre em três passos lógicos:

Passo 1: A Inicialização (Visita ao `DoHeader`) Quando o *codegen* encontra a declaração do ciclo (`DO 10 I = 1, 5`), ele executa as seguintes tarefas:

1. Avalia o valor inicial (1) e guarda-o na variável iteradora `I`.
2. Geração de Nomes: O compilador gera as *strings* dos nomes dos *labels* em memória (ex: `"DOSTART10"` e `"DOEND10"`).
3. Âncora de Início: Emite a marcação física de início no código: `DOSTART10:`.
4. Referência Antecipada: Emite o código de verificação de limite (`I > 5`). Se a condição for verdadeira, a máquina usa a instrução `jz DOEND10` para saltar.
5. O Registo de Estado: O compilador grava no dicionário `loop_contexts[10]` as informações que precisará no futuro: o nome da variável (`I`), e os nomes das *strings* dos *labels* de início e fim.

Passo 2: O Corpo do Ciclo O gerador continua a sua travessia linear da AST. Encontra o nó `Print` e emite as instruções normais de escrita na consola, completamente alheio ao facto de estar “dentro” de um ciclo.

Passo 3: O Fecho e Retorno (Visita ao `Continue`) Quando o *codegen* chega ao nó `Continue` com o *label* 10, a ligação é restabelecida:

1. O *codegen* interroga o dicionário `loop_contexts` perguntando pelo contexto do ciclo 10.
2. Sabendo agora que a variável de controlo é o `I`, o compilador emite as instruções para a incrementar (`pushg I, pushi 1, add, storeg I`).
3. Emite o salto de retorno incondicional de volta ao topo (`jump DOSTART10`).
4. A Âncora de Saída: Por fim, o compilador “planta” fisicamente a porta de saída no código inserindo o *label* final (`DOEND10:`). É exatamente neste ponto do ficheiro que a instrução `jz` gerada no Passo 1 vai “aterrar” quando a variável `I` ultrapassar o valor de 5.

Esta abordagem mantém a Árvore Sintática Abstrata puramente sequencial, e delega a complexidade de fecho de escopos e gestão de memória iterativa inteiramente para o gerador de código, resultando numa arquitetura robusta e livre de conflitos de *parsing*.

7.5. Integração Nativa de Funções Intrínsecas

O compilador efetua o mapeamento direto das Funções Intrínsecas da linguagem (*Intrinsic Functions*) detetadas no nó `ArrayRef` para os respetivos opcodes da EWVM, nomeadamente:

- Aritmética modular (`MOD`) mapeada diretamente para `mod`.
- Funções Trigonómicas (`SIN`, `COS`) mapeadas para `fsin` e `fcos`.
- Operações de Conversão Explícita de Tipo (`INT`, `REAL`) mapeadas para `ftoi` e `itof`.

8. Conclusão

O compilador apresentado satisfaz os requisitos estabelecidos para a disciplina de Processamento de Linguagens. O projeto implementa também passagens de otimização estática que garantem eficiência espacial e temporal. Gostaríamos de ter conseguido implementar sub-rotinas dinâmicas (*SUBROUTINE* e *FUNCTION*) mas ainda assim estamos contentes com o resultado obtido. A implementação deste compilador, apesar da sua simplicidade relativa, permitiu compreender melhor a enorme complexidade envolvida no desenvolvimento de compiladores modernos como o GCC, GHC, LLVM/Clang, rustc ou mesmo motores JIT como o V8. O facto de ferramentas desta dimensão e sofisticação serem desenvolvidas e disponibilizadas gratuitamente torna ainda mais impressionante o trabalho das comunidades e equipas responsáveis por estes projetos.